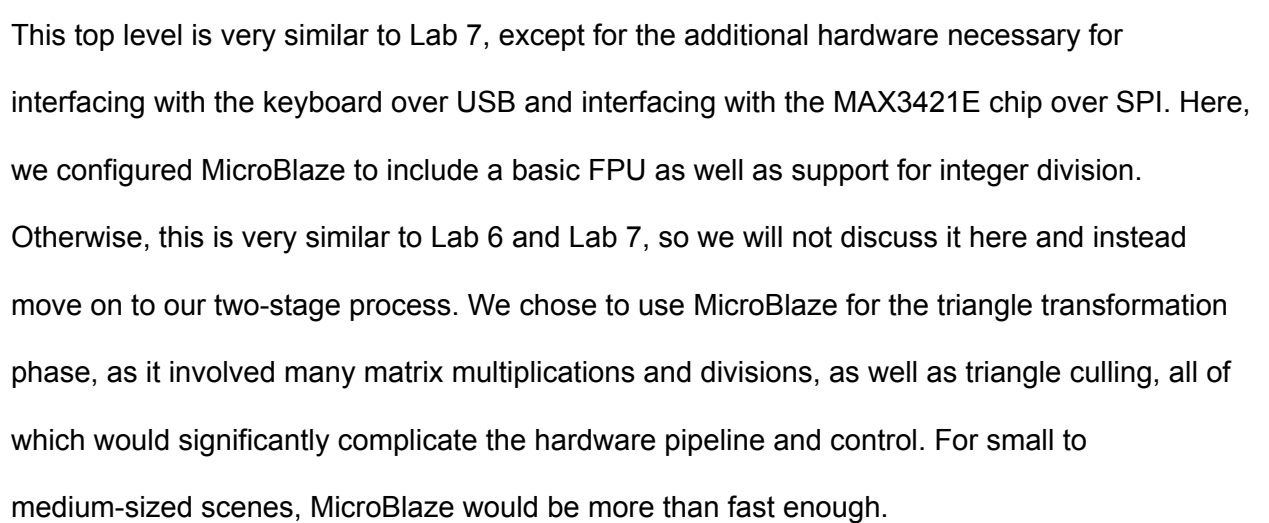




Triangle Transformation

All of the supporting math was learned from [Scratchapixel](#) and [ogldev.org](#).

We will walk through the theory behind the C code that implements the triangle transformation.

We define our directions as follows: +x is right, +y is up, and +z is into the screen. The first step is the transformation from world space into camera space by the view matrix:

```
float sin_yaw = sin_lookup(yaw);
float cos_yaw = cos_lookup(yaw);
float tx = -(cos_yaw * cam_x + sin_yaw * cam_z);
float ty = -cam_y;
float tz = -(-sin_yaw * cam_x + cos_yaw * cam_z);

const float view_mat[16] = {cos_yaw, 0.0f, sin_yaw, tx,
                             0.0f, 1.0f, 0.0f, ty,
                             -sin_yaw, 0.0f, cos_yaw, tz,
                             0.0f, 0.0f, 0.0f, 1.0f};
```

This matrix, although representing 3d coordinates, is a 4x4 matrix. This is so that translation and rotation can all be done by one matrix multiplication. When 3d vertices are loaded from the mesh, they are given a 4th w coordinate (remember this, it will be used later), which is just 1.

Thus, the transformed vertex will have tx, ty, and tz added to its correct dimensions, respectively.

The upper 3x3 portion of this matrix is simply a clockwise rotation about the y-axis (up and down in screen space). The reason why tx, ty, and tz are negative, as well as the fact that the rotation is clockwise rather than counterclockwise, is that this matrix is the inverse of the matrix that moves the camera. For example, if we want to move the camera 5 units to the right and rotate it counterclockwise by 90 degrees, we instead move the entire world left 5 units and rotate the world 90 degrees clockwise.

The next step is transforming from camera space into clip space through perspective projection:

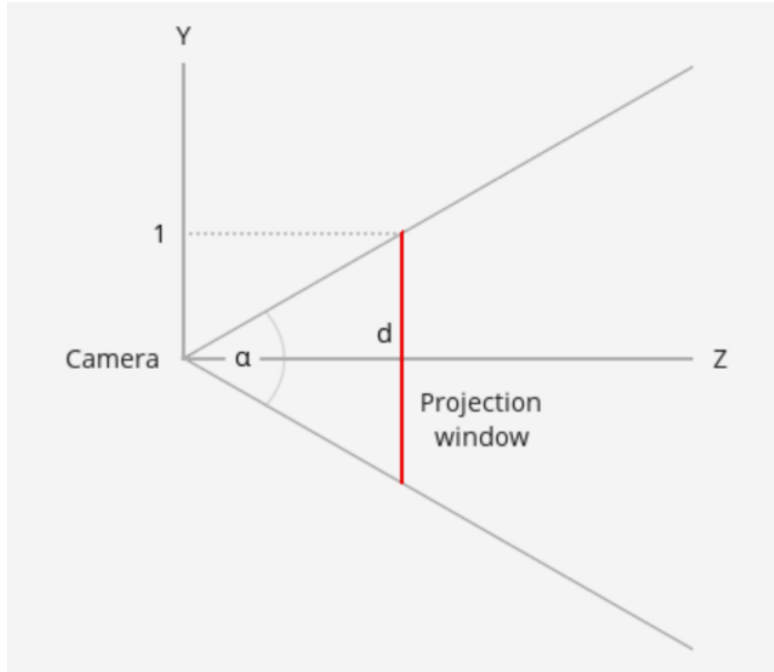
```
// ===== PROJECTION MATRIX (for [0, 1] depth) =====
// Pre-computed for:
// - FOV: 60 degrees
// - Aspect ratio: 320/240 = 4/3
// - Near plane: 1.0
```

```

// - Far plane: 300.0 (to see entire Cornell box at z=0..255)
//
// Formula:
// f = 1/tan(60°/2) = 1/tan(30°) ≈ 1.732
// m[0][0] = f/aspect = 1.732 / (4/3) = 1.299
// m[1][1] = f = 1.732
// m[2][2] = far/(far-near) = 300/(300-1) ≈ 1.003
// m[2][3] = -(far*near)/(far-near) = -300*1/299 ≈ -1.003
// m[3][2] = 1.0 (for [0,1] depth, not -1.0)
// Matrix generated by AI
// Asked Claude: "Generate a fixed projection matrix assuming I
only output to a 320x240 display, my far plane is at z = 300, and FOV = 60"
const float proj_mat[16] = {1.299f, 0.0f, 0.0f, 0.0f,
                             0.0f, 1.732f, 0.0f, 0.0f,
                             0.0f, 0.0f, 1.003f, -1.003f,
                             0.0f, 0.0f, 1.0f, 0.0f};

```

The role of this matrix is to create a foreshortening effect – objects further away from the camera should look smaller, while objects closer to the camera should look bigger. We can also adjust how much fits into our view, or our Field of View (FOV). We also account for the aspect ratio by scaling the x-axis. We create foreshortening by projecting all vertices onto a 2x2 projection window, as shown below. Note to include more in our FOV, we effectively move further away or closer to the projection window (as opposed to moving the projection window, which would change its dimensions). Thus, our first step is calculating the distance from the camera to the projection window, d . $\tan(\frac{\alpha}{2}) = \frac{1}{d}$, so $d = \frac{1}{\tan(\frac{\alpha}{2})}$.

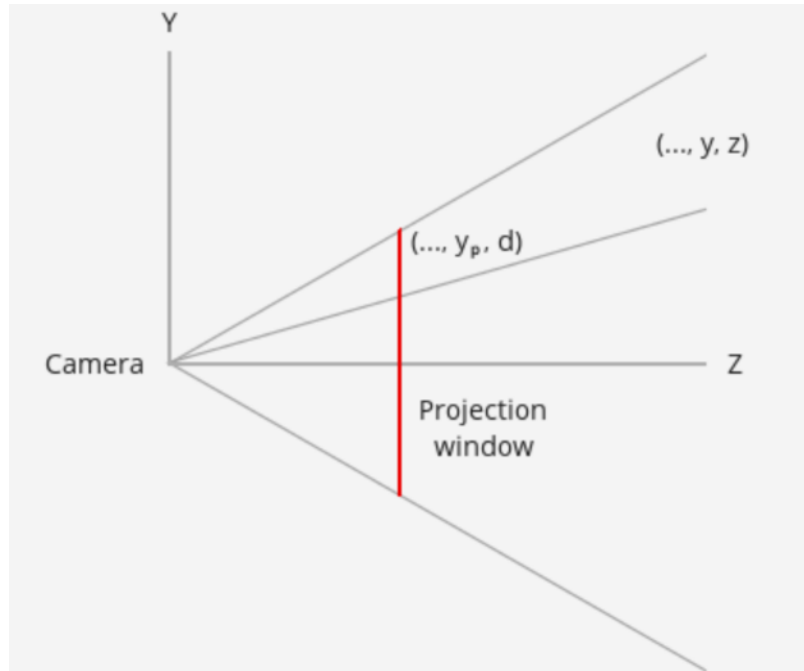


From <https://ogldev.org/www/tutorial12/tutorial12.html>

Now let us find y' , the projected y . We can use similar triangles to calculate y' . We can see the small triangle below with length d and height y' , but let us also imagine a bigger triangle with length z and height y . These triangles are similar, so $\frac{y'}{d} = \frac{y}{z}$, so $y' = \frac{yd}{z} = \frac{y}{\tan(\frac{\alpha}{2})z}$. We can do

the exact same for the x coordinate, except we will additionally divide by the aspect ratio to effectively “squeeze” the x -axis by scaling it to fit more x space than y space (assuming an aspect ratio greater than 1), thus $x' = \frac{xd}{z} = \frac{x}{\tan(\frac{\alpha}{2})az}$, where a is the aspect ratio. Dividing by z

is commonly known as the perspective divide.



From <https://ogldev.org/www/tutorial12/tutorial12.html>

$\tan(\frac{\alpha}{2})a$ is a constant, as we know the FOV and aspect ratio (4/3 for 320x240 resolution) before each frame. However, we must divide by z , and this cannot be encoded in a matrix; thus, we divide afterwards.

Now we have our x and y coordinates in normalized coordinates from -1 to 1. However, we want to divide by z afterwards. This was typically hardcoded to happen in the GPU for the x , y , and z coordinates. This meant that the z coordinate would be 1 for all vertices; thus, we can store the original z in w by storing 1 in position [3,2] of the matrix. We also want to store a normalized z in the z coordinate. We don't want to draw objects too close to the camera as this would result in z -fighting, which could create camera inconsistencies. We can also cull objects that are very far away to improve performance. Thus, we define a near plane and a far plane and then solve for a function $f(z) = A + \frac{B}{z}$, where A is in position [2, 2] in the matrix and B is in position [2,3].

This function accounts for the perspective divide in z .

These 2 matrices represent most of the conceptual difficulty of the transformation. Thus, the rest of the code will be explained briefly. Now that we have these 2 matrices, we can compose them together and multiply them by every vertex. After multiplying, we can check that each vertex is within our view (known as frustum culling). If we had already divided by w (the original z value), this would be checking that x and y are in $[-1, 1]$, and that z is in $[0, 1]$, but since we haven't yet, we compare with w instead. If the triangle is outside our viewing space, we don't send it, further improving performance. We also check if w is very small and skip the triangle if it is. This signifies the object is very close to the near plane, and dividing by 0 will create undefined or infinite values. Now we do the perspective divide and then remap our normalized ranges into $[0, 320]$ and $[0, 240]$ for our x and y values. We also multiply z by 255 and store all these values as integers to avoid floating-point computation in hardware. We also provide the reciprocal of the area to the hardware for calculating barycentric coordinates, which will be used later. This is calculated by calculating two vertices representing two edges of the triangle, and then taking their cross product and dividing by 2, and finally taking the reciprocal. We used memory-mapped I/O to write these values over AXI.

Rasterization Hardware

AXI:

The first step of drawing triangles is to properly receive them from MicroBlaze over AXI. For this, we use a modified implementation of Lab 7.1 AXI and a FIFO. From MicroBlaze, we only write 32-bit words, so we don't implement write strobe logic. In fact, we only use 6 addresses, which we write to sequentially from MicroBlaze. We use a buffer to store the writes to addresses 0-4. Then, on the 5th write, which writes to address 5, we pass the buffer contents along with the current write data into a FIFO, which holds the complete triangle data.

AXI Receives Data:

```
slv_regs[axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]][(byte_index*8) +: 8] <=
S_AXI_WDATA[(byte_index*8) +: 8];
//end
fifo_wr_en <= 1'b0;
if (axi_awaddr[4:2] == 3'd5 && !fifo_full) begin
    fifo_wr_en <= 1'b1;
    fifo_din <= {
        S_AXI_WDATA, // r_area
        slv_regs[4], // color + v3z
        slv_regs[3], // v3y + v3x
        slv_regs[2], // v2z + v2y
        slv_regs[1], // v2x + v1z
        slv_regs[0] // v1y + v1x
    };
end
```

The FIFO prevents the rest of the hardware from dropping triangles when it isn't ready to process them. We implemented a handshake between the first pipeline stage of rasterization and the FIFO. The FIFO is read when the raster pipeline is ready, and the FIFO was not empty on the last cycle (as reads take one cycle).

This triangle data is then processed, and the packet is unpacked and dispersed to the respective modules. In the C code, we assemble a triangle packet that is 192 bits long. Once it is transmitted through AXI, we can decode it.

C Packet Construction:

```
typedef struct {
    uint32_t v0v1; // Maps to lower 16 bits of addr[0]
    uint32_t v2v3; // Maps to lower 16 bits of addr[1]
    uint32_t v4v5; // Maps to upper 16 bits of addr[1]
    uint32_t v6v7; // Maps to lower 16 bits of addr[2]
    uint32_t v8color; // Maps to upper 16 bits of addr[3]
    int32_t r_area; // Maps to addr[5]
} TrianglePacket;
```

System Verilog Deconstruction:

```
assign inv_area = fifo_dout[191:160];
assign color = fifo_dout[151:144];
assign z3 = fifo_dout[143:128];
assign v3y = fifo_dout[119:112];
assign v3x = fifo_dout[104:96];
assign z2 = fifo_dout[95:80];
assign v2y = fifo_dout[71:64];
assign v2x = fifo_dout[56:48];
assign z1 = fifo_dout[47:32];
assign v1y = fifo_dout[23:16];
assign v1x = fifo_dout[8:0];
```

Top Level Controller:

We then enter the triangle processing pipeline. The entry into the pipeline begins with our highest-level triangle controller. This state machine produces the triangle ready signal and also checks for a triangle valid signal. The handshake occurs when both signals are on, and triangle ready is immediately deasserted 1 cycle later. This allows the FIFO read signal to be on for a single clock cycle, only popping a single triangle at a time.

We always automatically clear all 3 of our buffers from start to finish before any processing.

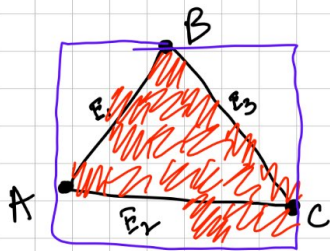
Specifically, our design contains 3 buffers: a double frame buffer, each with an 8-bit width and 76,800 depth, and a 16-bit wide z-buffer with a depth of 76800.

Our clear buffer state resets the values of each of these buffers, setting both frame buffers to x00, and setting the z-buffer to 0xFF. This is necessary to do once every frame, otherwise resulting in trails and inaccurate depth information between frames. We must treat each frame as a completely new scene from the hardware point of view, which also means we must send new triangle data every frame from the microblaze.

Once the buffer clearing has finished, resetting the values of all 3 buffers, we set a special flip-flop that tracks whether we have completed this action to 1. This way, we will not be flagged to clear the buffers again until we have another vsync falling edge.

We can then move into the rasterization pipeline. This state machine runs once for each triangle. After drawing a full triangle, we go back to waiting for another one.

Early plans for rasterization pipeline:



$$(v_1x - v_3x)(v_2y - v_3y) - (v_2x - v_3x)(v_1y - v_3y)$$

Start! $\rightarrow$ S1: $prod1 = a_1 \cdot b_{bx_i}$

$prod2 = \dots$

$prod6 = \dots$

S2: $E_1 = prod1 + prod2 + c_1$

$\vdots$
 $E_3 = \dots$

S3: $E_{1-r} = E_1$
 $\vdots$
 $E_{3-r} = E_3$ Once per y loop

S4: $E_{1-r} += a_1$

$\vdots$
 $E_{3-r} += a_3$

IF $E_{i-r} > 0$, (zbuf)? print : cont.

S5: $E_1 += b_1$

$\vdots$
 $E_3 += b_3$

Done

zbuffer:

$z = ?$

if ($z < z_{\text{buf}}[\text{pixel}]$) return true

else return false.

Interpolation logic:

$$S1: \text{Barycentric-}w_1 = E_{1-\text{row}} \cdot \text{inv-area}$$

$\vdots$

$$w_3 = E_{3-\text{row}} \cdot \text{inv-area}.$$

$$S2: z = w_1 \times z_1 + w_2 \times z_2 + w_3 \times z_3$$

$$S3: z_{\text{buff-addr}} = y \times 320 + x$$

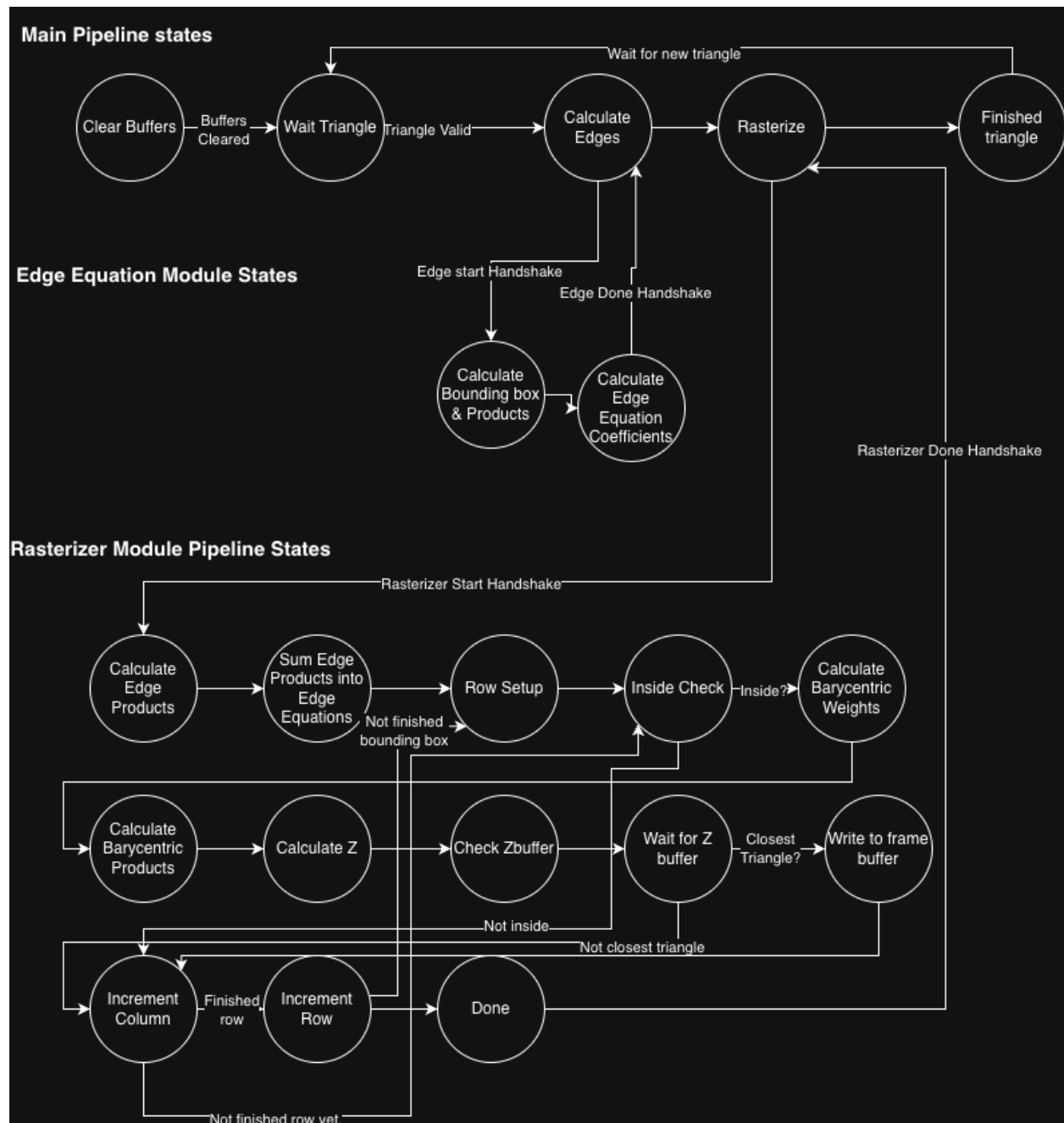
$$\text{addr-gpu} = y \times 320 + x$$

S4: if ($z_{\text{buff-dot}} > z$): $\text{fb-din} = \text{Color}$.

else skip writing

always transition to done.

State diagram:



Rasterization Part 1: Edge detection (Stages 1 and 2):

To begin drawing triangles, we first need a way to figure out whether a particular pixel is inside a triangle. We also need to know which pixels are even worth checking to see if they're in the triangle, as it would be an incredible waste of time and effort to check every pixel on the screen.

To solve both these problems, the first step of drawing a triangle is to create the bounding box and calculate the coefficients for the triangle edges.

The pipeline controller spends 3 clock cycles in this module. The first cycle is spent latching inputs into the module such that we can support both combinational and sequential calculations with the same data. In the next two cycles, we perform the multiplication necessary to calculate our edge coefficients. Calculating the bounding box is combinational.

The bounding box is the smallest rectangular box that can fit the entire triangle inside. To calculate the bounding box, we find the smallest x value, smallest y value, biggest x value, and biggest y value out of all 3 vertices of the triangle. We also make sure to cull pixels that will be outside the 320x240 screen in this stage.

As an example, we show one 3-way comparison performed to find the smallest x value for the bounding box:

```
assign temp1 = (v1x < v2x) ? v1x : v2x;
assign bbxi = (temp1 < v3x) ? temp1 : v3x;
assign bbxi = (bbxi > 'd320) ? 'd320 : bbxi;
assign bbxi = (bbxi < 'd0) ? 'd0 : bbxi;
```

Once we have computed the bounding box, we must also compute the coefficients for edge equations.

A lot of the math used this stack overflow post as a basis:

<https://stackoverflow.com/questions/2049582/how-to-determine-if-a-point-is-in-a-2d-triangle>.

An edge equation looks like this:

$$\text{Edge}(x,y) = A*x + B*y + C$$

$$A = y_1 - y_2 = \text{delta } y.$$

$$B = x_2 - x_1 = \text{delta } x.$$

$$C = x_1*y_2 - x_2*y_1$$

So if $\text{Edge}(x,y) \geq 0$ then the pixel is inside. If all 3 edges are ≥ 0 , then we should draw the pixel.

Multiplications take multiple cycles. The best way to force the synthesis to use DSP slices for our multiplication is to add pipeline registers in the multiplication operation. We therefore split this into 2 stages and therefore made it a 2-clock-cycle pipelined process.

The sequential calculations performed in the module are below:

Edge v1 to v2.

$a1 = v1y - v2y$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$b1 = v2x - v1x$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$c1 = v1x \cdot v2y - v2x \cdot v1y$, Takes 2 clock cycles. We do it in 2 stages & handshake.

Edge v2 to v3.

$a2 = v2y - v3y$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$b2 = v3x - v2x$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$c2 = v2x \cdot v3y - v3x \cdot v2y$, Takes 2 clock cycles. We do it in 2 stages & handshake.

Edge v3 to v1.

$a3 = v3y - v1y$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$b3 = v1x - v3x$, Takes 1 clock cycle. We do it combinational after latching the inputs.

$c3 = v3x \cdot v1y - v1x \cdot v3y$, Takes 2 clock cycles. We do it in 2 stages & handshake.

Similar to the top-level triangle controller, the edge equations module also has two handshaking signals so that calculations begin and end on time, therefore passing accurate values to the next step in the pipeline. To achieve this, we created "edge_start" and "edge_done." Start is asserted by the top-level triangle controller once buffers are cleared and all required information is retrieved from AXI. The edge equations module is then able to latch the correct information and begin processing it. Inside the module, we have 2 logic signals to keep track of the linear

pipeline: ready_s1, ready_s2. The following controller steps through our pipeline and asserts “edge_done” once the output information is correct:

```
always_ff @(posedge clk) begin
    if(rst) begin
        ready_s1 <= 0;
        ready_s2 <= 0;
    end else begin
        ready_s1 <= edge_start;
        ready_s2 <= ready_s1;
    end
end
assign edge_done = ready_s2;
```

Rasterization Part 2: Inside Check & Writing Pixels (Stages 3 to 12 (non-linear)):

Our computation then moves into the rasterizer module inside rasterizer.sv. This module contains a 13-state FSM with a non-linear pixel drawing pipeline. It takes the edge equation coefficients and bounding box from the previous module, as well as the inverse area and color from AXI as inputs. As output, it's able to supply frame buffer and z-buffer reading/writing signals. Therefore, the bulk of our hardware computation takes place inside this module.

Below is the pseudo-code for this module:

```
Calculate E1, E2, E3 a single time.
Edge(x,y) = A*x + B*y + C

PART 1:
E1 = a1*bbxi + b1*bbyi + c1
E2 = a2*bbxi + b2*bbyi + c2
E3 = a3*bbxi + b3*bbyi + c3

PART 2:
for(integer y = bbyi; y <= bbyf; y++) begin
    E1_row = E1;
    E2_row = E2;
    E3_row = E3;
    for(integer x = bbxi; x <= bbxf; x++) begin
        if(E1_row >= 0 && E2_row >= 0 && E3_row >= 0): CHECK ZBUFFER & Write color.
        E1_row += a1;
```

```

        E2_row += a2;
        E3_row += a3;
    end
    E1 += b1;
    E2 += b2;
    E3 += b3;
end

```

We first begin with a halt state that begins the handshaking process for the rasterization module.

The top-level triangle controller asserts "rasterizer_start" once data is ready from the edge module, and this triggers the halt state to move into the edge equation evaluation.

Like in the pseudo code, we must first compute all three of our edges.

This requires a multiplication, so we do this in 2 separate cycles: one to compute all 6 products, and another to sum them into our edges. We also must cast these edge coefficients to be signed such that our multiplications also end up signed.

```

prod1 <= $signed(a1) * $signed({1'b0, bbxi});
prod2 <= $signed(b1) * $signed({1'b0, bbyi});
prod3 <= $signed(a2) * $signed({1'b0, bbxi});
prod4 <= $signed(b2) * $signed({1'b0, bbyi});
prod5 <= $signed(a3) * $signed({1'b0, bbxi});
prod6 <= $signed(b3) * $signed({1'b0, bbyi});

```

And in another state:

```

e1 <= $signed(prod1) + $signed(prod2) + $signed(c1);
e2 <= $signed(prod3) + $signed(prod4) + $signed(c2);
e3 <= $signed(prod5) + $signed(prod6) + $signed(c3);

```

We can then begin computing pixels inside the bounding box. Just as in the loop, we must first iterate through each cell of each row, and then every row until we reach the bottom of the bounding box. To accomplish this in hardware, we cache our edge equations for the start of each row and increment it in another state called col_inc (short for column increment) until we reach the end of the row in the bounding box. We also increment between rows in a state called row_inc. These states ensure that we cover every pixel in the bounding box.

Inside this loop we must check whether a particular pixel is inside the triangle we are currently drawing or not. This check is made in the “inside_check” state, which takes the evaluated edge equations and checks whether they are all positive. This looks for clockwise winding of triangle vertices, which automatically completes backface culling. Therefore, if all the edges are positive, then we know that the pixel we are currently looking at is, in fact, inside the triangle we want to draw. If the vertices are arranged in a way that has counterclockwise winding, then we know that we must be looking at a face that is behind our main shape, so we do not draw it.

If we decide that this pixel is, in fact, inside our triangle, we must then check that the pixel wouldn't already be covered by a triangle that is closer. To check this, we use a z-buffer. But first, we must figure out which z value projects to the current pixel we are at.

To do this, we use barycentric coordinates. In essence, we must find out what the z value is for our x and y coordinates on the triangle. We do not have this information given from MicroBlaze, as this would require a bounding box and previous knowledge of what pixels map to what points on the triangle, at which point it might be easier to render everything in software. To do this accurately in hardware, barycentric coordinates take a weighted average of the z coordinates of each vertex and settle on a z value closer to the vertex closest to that point.

To achieve this in hardware, we multiply our inverse area by our edge equation to find a weight for each edge of our triangle. These come out to be 54-bit $w1_raw$, $w2_raw$, and $w3_raw$ signed values. We then multiply these by the corresponding z coordinates from each vertex to get 3 71-bit products. We sum these into a calculated z value for our pixel. Since the inverse area was a 32-bit value in 8.24 fixed-point format, we must sample only the bottom 8 bits of the non-fractional part and the top 8 bits of the fractional part of this value to go inside our z buffer. This allows plenty of room for intersection tracking down to very precise fractional z values, while also allowing us to track depth up to z values as large as 256.

When we have computed the z value for the triangle at this pixel, we can then compare it to the z value stored in the z-buffer. Since reading BRAM has a 1-cycle latency, we introduce a wait state. If our new z value is lower than the previous smallest z value in the buffer, then we should replace it and draw our pixel. If not, we move on to the next pixel.

If we decide that we should draw this pixel, we address our frame buffer and write the correct color for the triangle.

Module Descriptions

HDMI Controller Top Level (hdmi_top_level.sv):

Inputs: axi_aclk, axi_aresetn, [C_AXI_ADDR_WIDTH-1 : 0] axi_awaddr, [2 : 0] axi_awprot, axi_awvalid, [C_AXI_DATA_WIDTH-1 : 0] axi_wdata, [(C_AXI_DATA_WIDTH/8)-1 : 0] axi_wstrb, axi_wvalid, axi_bready, [C_AXI_ADDR_WIDTH-1 : 0] axi_araddr, [2 : 0] axi_arprot, axi_arvalid, axi_bready, [C_AXI_ADDR_WIDTH-1 : 0] axi_araddr, [2 : 0] axi_arprot, axi_arvalid, axi_rready
Outputs: axi_awready, axi_wready, [1 : 0] axi_bresp, axi_bvalid, axi_arready, [C_AXI_DATA_WIDTH-1 : 0] axi_rdata, [1 : 0] axi_rresp, axi_rvalid

Purpose: The purpose of this module is to act as the top-level module for the entire IP. Here we instantiate all of our sub-modules so that we can connect them all and get them working.

Description: This module contains instantiations of all other modules such as the framebuffer, rasterizer, AXI controller, and all of the other modules that make up the IP. We did not change this file other than removing the instantiation of font_rom from Lab 7.2.

VGA Controller (VGA_Controller.sv)

Inputs: pixel_clk, reset

Outputs: hs, vs, active_nblank, sync, [9:0] drawX, [9:0] drawY

Description: This module takes the pixel clock and generates an hsync signal, a vsync signal, and outputs the current DrawX and DrawY value that the color mapper should be outputting RGB values for.

Purpose: This module keeps all the display hardware in sync using the pixel clock. It also tells the color mapper when we've finished a row and are about to move to the next (hsync on, so draw blanks) or when we've finished a frame and are about to move to the next (vsync on, so draw blanks). It also tells the entire system which pixel we're currently drawing using the DrawX and DrawY values. This is unchanged from Lab 7.

HDMI Text Controller AXI Module (hdmi_top_level_axi.sv)

Inputs: vsync, [9:0] drawX, [9:0] drawY, S_AXI_ACLK, S_AXI_ARESETN, [C_S_AXI_ADDR_WIDTH-1 : 0] S_AXI_AWADDR, [2 : 0] S_AXI_AWPROT, [C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_WDATA, [(C_S_AXI_DATA_WIDTH/8)-1 : 0]

S_AXI_WSTRB, S_AXI_WVALID, S_AXI_BREADY, [C_S_AXI_ADDR_WIDTH-1 : 0]
S_AXI_ARADDR, [2 : 0] S_AXI_ARPROT, S_AXI_ARVALID, S_AXI_RREADY
Outputs: [3:0] red, [3:0] green, [3:0] blue, S_AXI_AWVALID, S_AXI_AWREADY,
S_AXI_WREADY, [1 : 0] S_AXI_BRESP, S_AXI_BVALID, S_AXI_ARREADY,
[C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_RDATA, [1 : 0] S_AXI_RRESP, S_AXI_RVALID,
Description: This module contains all the AXI host handshaking logic for reads and writes, as well as the functionality of the color mapper.

Purpose: This module is used to allow Microblaze to write into the FIFO with memory-mapped I/O. Additionally, this module provides RGB values to the VGA to HDMI module. We modified slv_regs to be only 8 registers, and added logic to have slv_regs input into the FIFO when appropriate. The colors are now simply read out of the framebuffer per pixel, and there are no palette registers. It also contains the main FSM for controlling the pipeline.

Edge Equation and Bounding Box Module (edge_eq_bb.sv)

Inputs: clk, rst, [8:0] v1x_in, [8:0] v2x_in, [8:0] v3x_in, [7:0] v1y_in, [7:0] v2y_in, [7:0] v3y_in, edge_start

Outputs: edge_done, signed [9:0] a1, signed [9:0] b1, signed [9:0] a2, signed [9:0] b2, signed [9:0] a3, signed [9:0] b3, signed [17:0] c1, signed [17:0] c2, signed [17:0] c3, [8:0] bbxi, [8:0] bbxf, [7:0] bbyi, [7:0] bbyf

Description: This module contains the logic for generating a bounding box and the 3 edge equations given a set of a triangle's vertices. It has a delay of one clock cycle.

Purpose: This module provides the pipeline pixels over. It turns from abstract triangle vertices into pixels, which may or may not need to be set to a certain color.

Framebuffer (framebuffer.sv):

Inputs: clk, vsync, rst, wea, [ADDR_WIDTH-1:0] addra, [7:0] dina, [ADDR_WIDTH-1:0] addrb

Outputs: [7:0] doubt, front

Description: This module instantiates and abstracts two memories to hold two buffers for double buffering. It swaps buffers on every vsync to allow us to start generating the next frame while the previous one is being read (i.e. write to the framebuffer outside of vblank).

Purpose: This module functions as the write buffer for our pipeline. It can be overwritten in the same area many times to allow for overlapping triangles (or even one triangle completely covering another). It also functions as the read buffer for vga controller.

Rasterizer (rasterizer.sv):

Inputs: clk, rst, [31:0] inv_area, [7:0] color, signed [9:0] a1, signed [9:0] b1, signed [9:0] a2, signed [9:0] b2, signed [9:0] a3, signed [9:0] b3, signed [17:0] c1, signed [17:0] c2, signed [17:0] c3, [8:0] bbxi, [8:0] bbxf, [7:0] bbyi, [7:0] bbyf, [15:0] z1, [15:0] z2, [15:0] z3, rasterizer_start, [7:0] zbuf_dout

Outputs: rasterizer_done, write_enable_gpu, [7:0] data_in_gpu, [16:0] addr_gpu, [16:0] zbuf_addr, [7:0] zbuf_din, zbuf_we

Description: This is the main rasterizing module, which loops through the bounding box and colors the triangle.

Purpose: The purpose of this module is to iterate through the bounding box and decide whether to draw each pixel. This module handles checking whether each pixel is inside the triangle, and whether it's already being covered by something else, before writing the correct color value to the frame buffer.

HDMI Text Controller (C file):

Description: This code contains the logic to transform the triangle mesh into screen space triangles.

Purpose: This code passes triangles to the rendering hardware through a FIFO, one triangle at a time.

HDMI Text Controller (H file):

Description: This module contains the (AI-generated Cornell Box) triangle mesh that we chose to use. It also contains function declarations and the sin lookup table.

Purpose: This allows us to easily set the triangle mesh.

Simulations

Triangle Transformation

The following outputs come from this testbench. It generates a wireframe image where the borders of the triangle are colored rather than the whole triangle for better visibility. This testbench is partially AI-generated. I asked Claude to generate a testbench that uses my Cornell Box mesh and the logic in my main function from `hdmi_text_controller.c` to create an output image. I ran this test using three different camera locations.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#include <string.h>

// Screen dimensions
#define SCREEN_WIDTH 320
#define SCREEN_HEIGHT 240

// Cornell Box mesh data
static const uint8_t cornell_box[][10] = {
    // Floor (white)
    {0,0,0, 255,0,0, 255,0,255, 0xFF},
```

```

{0,0,0, 255,0,255, 0,0,255, 0xFF},
// Ceiling (white)
{0,255,0, 255,255,255, 255,255,0, 0xFF},
{0,255,0, 0,255,255, 255,255,255, 0xFF},
// Left wall (red)
{0,0,0, 0,0,255, 0,255,255, 0xE0},
{0,0,0, 0,255,255, 0,255,0, 0xE0},
// Right wall (green)
{255,0,0, 255,255,0, 255,255,255, 0x1C},
{255,0,0, 255,255,255, 255,0,255, 0x1C},
// Back wall (white)
{0,0,255, 255,0,255, 255,255,255, 0xFF},
{0,0,255, 255,255,255, 0,255,255, 0xFF},
// Small cube (white)
{51,0,102, 102,0,102, 102,0,153, 0xFF},
{51,0,102, 102,0,153, 51,0,153, 0xFF},
{51,51,102, 102,51,153, 102,51,102, 0xFF},
{51,51,102, 51,51,153, 102,51,153, 0xFF},
{51,0,102, 51,51,102, 102,51,102, 0xFF},
{51,0,102, 102,51,102, 102,0,102, 0xFF},
{51,0,153, 102,0,153, 102,51,153, 0xFF},
{51,0,153, 102,51,153, 51,51,153, 0xFF},
{51,0,102, 51,0,153, 51,51,153, 0xFF},
{51,0,102, 51,51,153, 51,51,102, 0xFF},
{102,0,102, 102,51,102, 102,51,153, 0xFF},
{102,0,102, 102,51,153, 102,0,153, 0xFF},
// Tall cube (white)
{153,0,179, 204,0,179, 204,0,230, 0xFF},
{153,0,179, 204,0,230, 153,0,230, 0xFF},
{153,128,179, 204,128,230, 204,128,179, 0xFF},
{153,128,179, 153,128,230, 204,128,230, 0xFF},
{153,0,179, 153,128,179, 204,128,179, 0xFF},
{153,0,179, 204,128,179, 204,0,179, 0xFF},
{153,0,230, 204,0,230, 204,128,230, 0xFF},
{153,0,230, 204,128,230, 153,128,230, 0xFF},
{153,0,179, 153,0,230, 153,128,230, 0xFF},
{153,0,179, 153,128,230, 153,128,179, 0xFF},
{204,0,179, 204,128,179, 204,128,230, 0xFF},
{204,0,179, 204,128,230, 204,0,230, 0xFF},
};

#define cornell_box_triangle_count (sizeof(cornell_box) /
sizeof(cornell_box[0]))

// Framebuffer (ASCII art representation)
char framebuffer[SCREEN_HEIGHT][SCREEN_WIDTH];

// Matrix and vector math
void matmul4x4(const float in1[16], const float in2[16], float out[16]) {
    for (int r = 0; r < 4; r++) {

```

```

        for (int c = 0; c < 4; c++) {
            float dot = 0.0f;
            for (int k = 0; k < 4; k++) {
                dot += in1[4 * r + k] * in2[4 * k + c];
            }
            out[4 * r + c] = dot;
        }
    }
}

void matvec4x1(const float mat[16], const float vec[4],
               float *out_vec) {
    for (int mat_r = 0; mat_r < 4; mat_r++) {
        float dot = 0.0f;

        for (int mat_c = 0; mat_c < 4; mat_c++) {
            dot += mat[4 * mat_r + mat_c] * vec[mat_c];
        }

        out_vec[mat_r] = dot;
    }
}

// Clear framebuffer
void clear_framebuffer() {
    for (int y = 0; y < SCREEN_HEIGHT; y++) {
        for (int x = 0; x < SCREEN_WIDTH; x++) {
            framebuffer[y][x] = ' ';
        }
    }
}

// Draw a pixel (ASCII art)
void draw_pixel(int x, int y, char c) {
    if (x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT) {
        framebuffer[y][x] = c;
    }
}

// Simple line drawing (Bresenham)
void draw_line(int x0, int y0, int x1, int y1, char c) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = x0 < x1 ? 1 : -1;
    int sy = y0 < y1 ? 1 : -1;
    int err = dx - dy;

    while (1) {
        draw_pixel(x0, y0, c);

```

```

        if (x0 == x1 && y0 == y1) break;
        int e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
    }
}

// Draw wireframe triangle
void draw_triangle(int x1, int y1, int x2, int y2, int x3, int y3, uint8_t
color) {
    // Determine character based on color
    char c = '.';
    if (color == 0xE0) c = 'R'; // Red
    else if (color == 0x1C) c = 'G'; // Green
    else c = '*'; // White

    draw_line(x1, y1, x2, y2, c);
    draw_line(x2, y2, x3, y3, c);
    draw_line(x3, y3, x1, y1, c);
}

// Display framebuffer (scaled down for terminal)
void display_framebuffer() {
    printf("\033[2J\033[H"); // Clear screen and move cursor to top

    // Display every 4th row and 2nd column (80x60 roughly fits terminal)
    for (int y = 0; y < SCREEN_HEIGHT; y += 4) {
        for (int x = 0; x < SCREEN_WIDTH; x += 2) {
            putchar(framebuffer[y][x]);
        }
        putchar('\n');
    }
}

// Save as PPM image
void save_ppm(const char* filename) {
    FILE* f = fopen(filename, "wb");
    fprintf(f, "P6\n%d %d\n255\n", SCREEN_WIDTH, SCREEN_HEIGHT);

    for (int y = 0; y < SCREEN_HEIGHT; y++) {
        for (int x = 0; x < SCREEN_WIDTH; x++) {
            char c = framebuffer[y][x];
            uint8_t r = 0, g = 0, b = 0;

            if (c == 'R') { r = 255; } // Red
            else if (c == 'G') { g = 255; } // Green
            else if (c != ' ') { r = g = b = 200; } // White/gray

            fputc(r, f); fputc(g, f); fputc(b, f);
        }
    }
}

```

```

    }
}
fclose(f);
printf("Saved to %s\n", filename);
}

int main() {
    // Camera parameters
    float cam_x = 127.5f, cam_y = 127.5f, cam_z = -20.0f;
    float yaw = 3.1415f / 12;

    printf("3D Renderer Test\n");
    printf("=====\n");
    printf("Camera: (%.1f, %.1f, %.1f), Yaw: %.2f rad\n\n", cam_x, cam_y,
cam_z, yaw);

    // View matrix
    float cos_yaw = cosf(yaw);
    float sin_yaw = sinf(yaw);
    float tx = -(cos_yaw * cam_x + sin_yaw * cam_z);
    float ty = -cam_y;
    float tz = -(-sin_yaw * cam_x + cos_yaw * cam_z);

    const float view_mat[16] = {
        cos_yaw,  0.0f, sin_yaw, tx,
        0.0f,     1.0f, 0.0f,  ty,
        -sin_yaw, 0.0f, cos_yaw, tz,
        0.0f,     0.0f, 0.0f,  1.0f
    };

    // Projection matrix (60° FOV, 4:3 aspect, near=1, far=300)
    const float proj_mat[16] = {
        1.299f,  0.0f,  0.0f,  0.0f,
        0.0f,    1.732f, 0.0f,  0.0f,
        0.0f,    0.0f,  1.003f, -1.003f,
        0.0f,    0.0f,  1.0f,   0.0f
    };

    // Combined MVP
    float mvp[16];
    matmul4x4(proj_mat, view_mat, mvp);

    // Clear framebuffer
    clear_framebuffer();

    // Render statistics
    int triangles_rendered = 0;
    int triangles_culled = 0;

```



```

// Process each triangle
for (int i = 0; i < cornell_box_triangle_count; i++) {
    // World space vertices
    float world_v1[4] = {(float)cornell_box[i][0],
(float)cornell_box[i][1],
(float)cornell_box[i][2], 1.0f};
    float world_v2[4] = {(float)cornell_box[i][3],
(float)cornell_box[i][4],
(float)cornell_box[i][5], 1.0f};
    float world_v3[4] = {(float)cornell_box[i][6],
(float)cornell_box[i][7],
(float)cornell_box[i][8], 1.0f};

    float clip1[4], clip2[4], clip3[4];
    matvec4x1(mvp, world_v1, clip1);
    matvec4x1(mvp, world_v2, clip2);
    matvec4x1(mvp, world_v3, clip3);

    // Frustum culling
    int all_left = (clip1[0] < -clip1[3] && clip2[0] < -clip2[3] &&
clip3[0] < -clip3[3]);
    int all_right = (clip1[0] > clip1[3] && clip2[0] > clip2[3] &&
clip3[0] > clip3[3]);
    int all_bottom = (clip1[1] < -clip1[3] && clip2[1] < -clip2[3] &&
clip3[1] < -clip3[3]);
    int all_top = (clip1[1] > clip1[3] && clip2[1] > clip2[3] &&
clip3[1] > clip3[3]);
    int all_near = (clip1[2] < 0 && clip2[2] < 0 && clip3[2] < 0);
    int all_far = (clip1[2] > clip1[3] && clip2[2] > clip2[3] &&
clip3[2] > clip3[3]);

    if (all_left || all_right || all_bottom || all_top || all_near ||
all_far) {
        triangles_culled++;
        continue;
    }

    // Check for degenerate w
    if (clip1[3] <= 0.0001f || clip2[3] <= 0.0001f || clip3[3] <=
0.0001f) {
        triangles_culled++;
        continue;
    }

    // Perspective divide
    float x1_ndc = clip1[0] / clip1[3];
    float y1_ndc = clip1[1] / clip1[3];
    float x2_ndc = clip2[0] / clip2[3];
    float y2_ndc = clip2[1] / clip2[3];

```

```

float x3_ndc = clip3[0] / clip3[3];
float y3_ndc = clip3[1] / clip3[3];

// Convert to screen coordinates
int x1_screen = (int)((x1_ndc + 1.0f) * 160.0f);
int y1_screen = (int)((1.0f - y1_ndc) * 120.0f);
int x2_screen = (int)((x2_ndc + 1.0f) * 160.0f);
int y2_screen = (int)((1.0f - y2_ndc) * 120.0f);
int x3_screen = (int)((x3_ndc + 1.0f) * 160.0f);
int y3_screen = (int)((1.0f - y3_ndc) * 120.0f);

// Draw triangle
uint8_t color = cornell_box[i][9];
draw_triangle(x1_screen, y1_screen, x2_screen, y2_screen,
              x3_screen, y3_screen, color);

    triangles_rendered++;
}

// Display results
display_framebuffer();

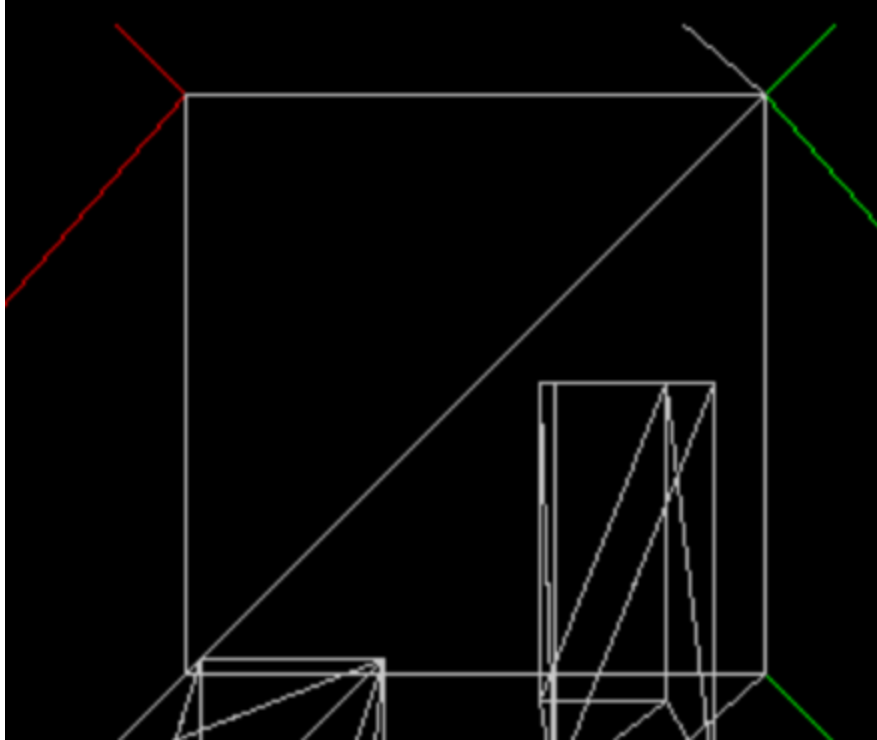
printf("\nStatistics:\n");
printf("  Triangles rendered: %d\n", triangles_rendered);
printf("  Triangles culled: %d\n", triangles_culled);
printf("  Total triangles: %d\n", cornell_box_triangle_count);

// Save to image file
save_ppm("output.ppm");

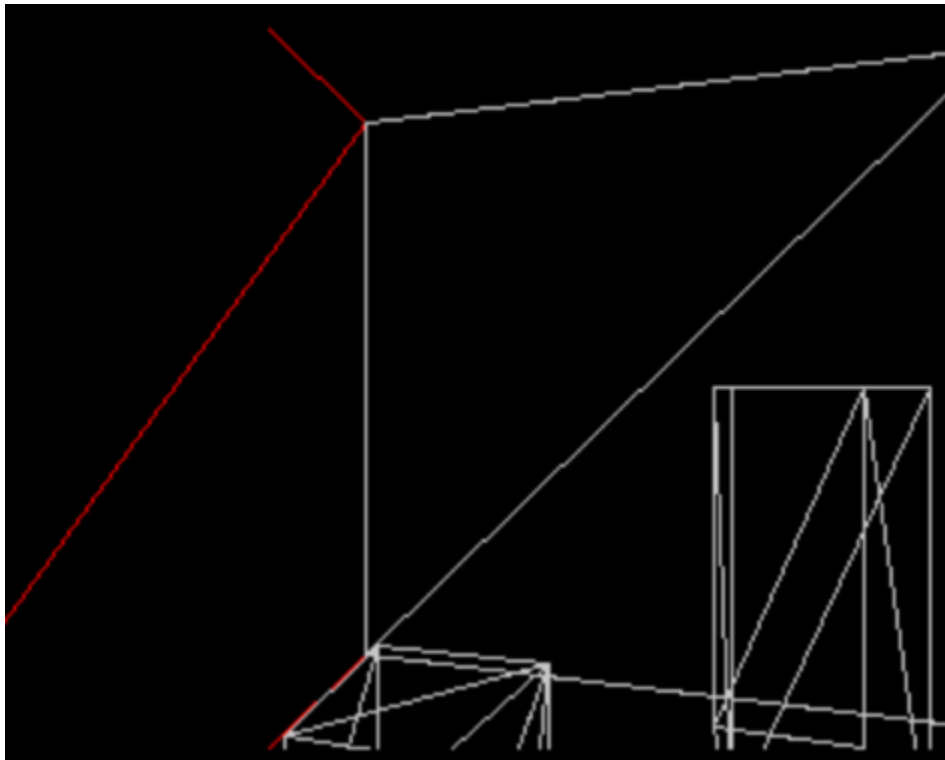
return 0;
}

```

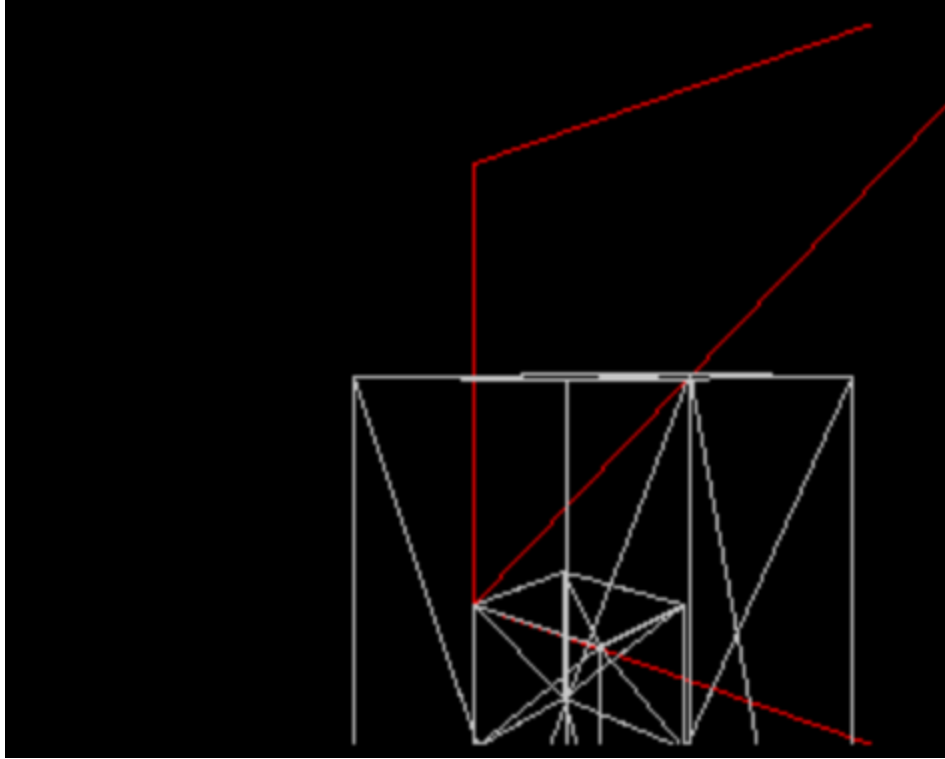
cam_x = 127.5, cam_y = 127.5, cam_z = -20, yaw = 0



cam_x = 127.5, cam_y = 127.5, cam_z = -20, yaw = $\pi/12$



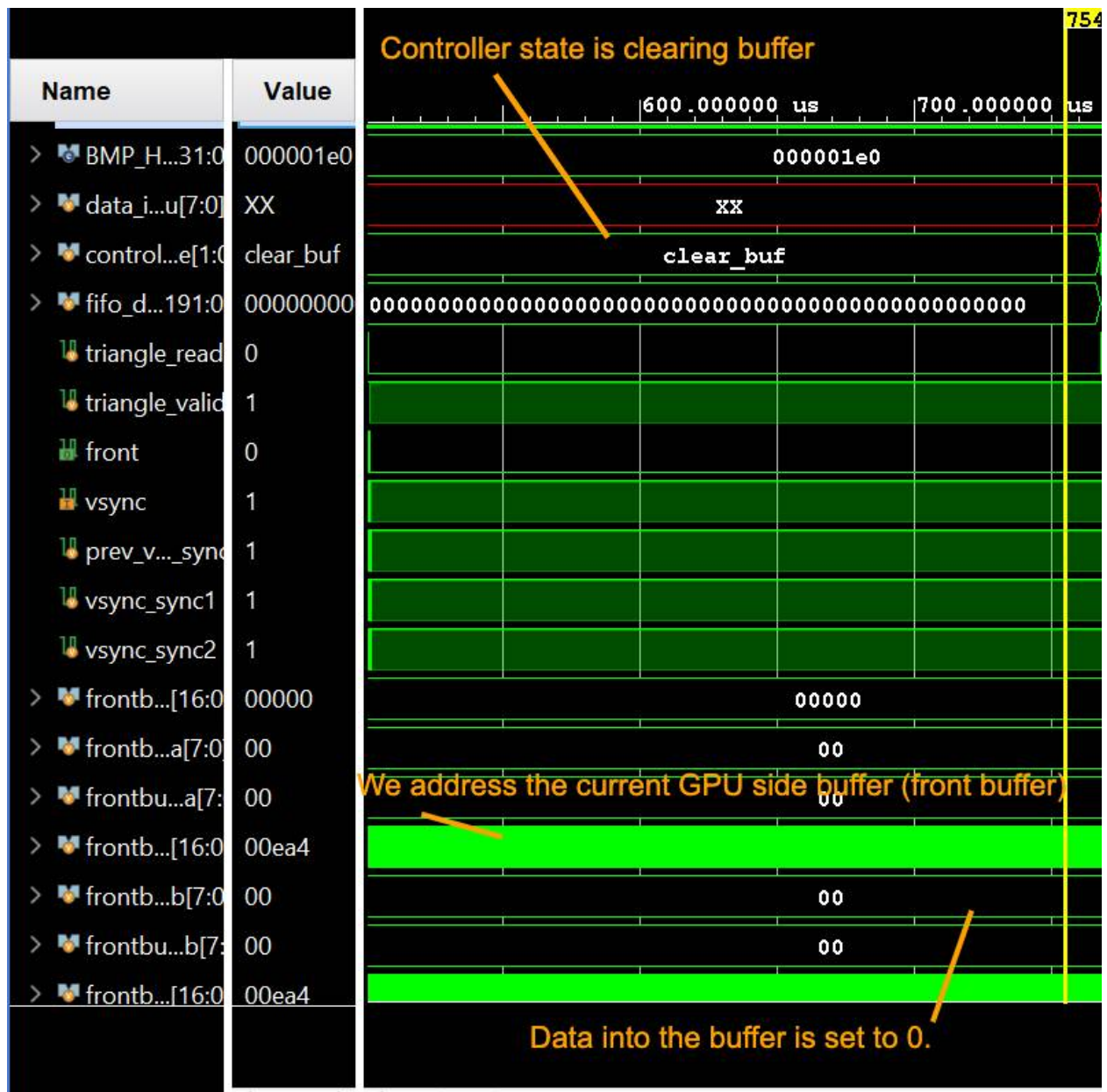
cam_x = 127.5, cam_y = 127.5, cam_z = -20, yaw = $3\pi/4$



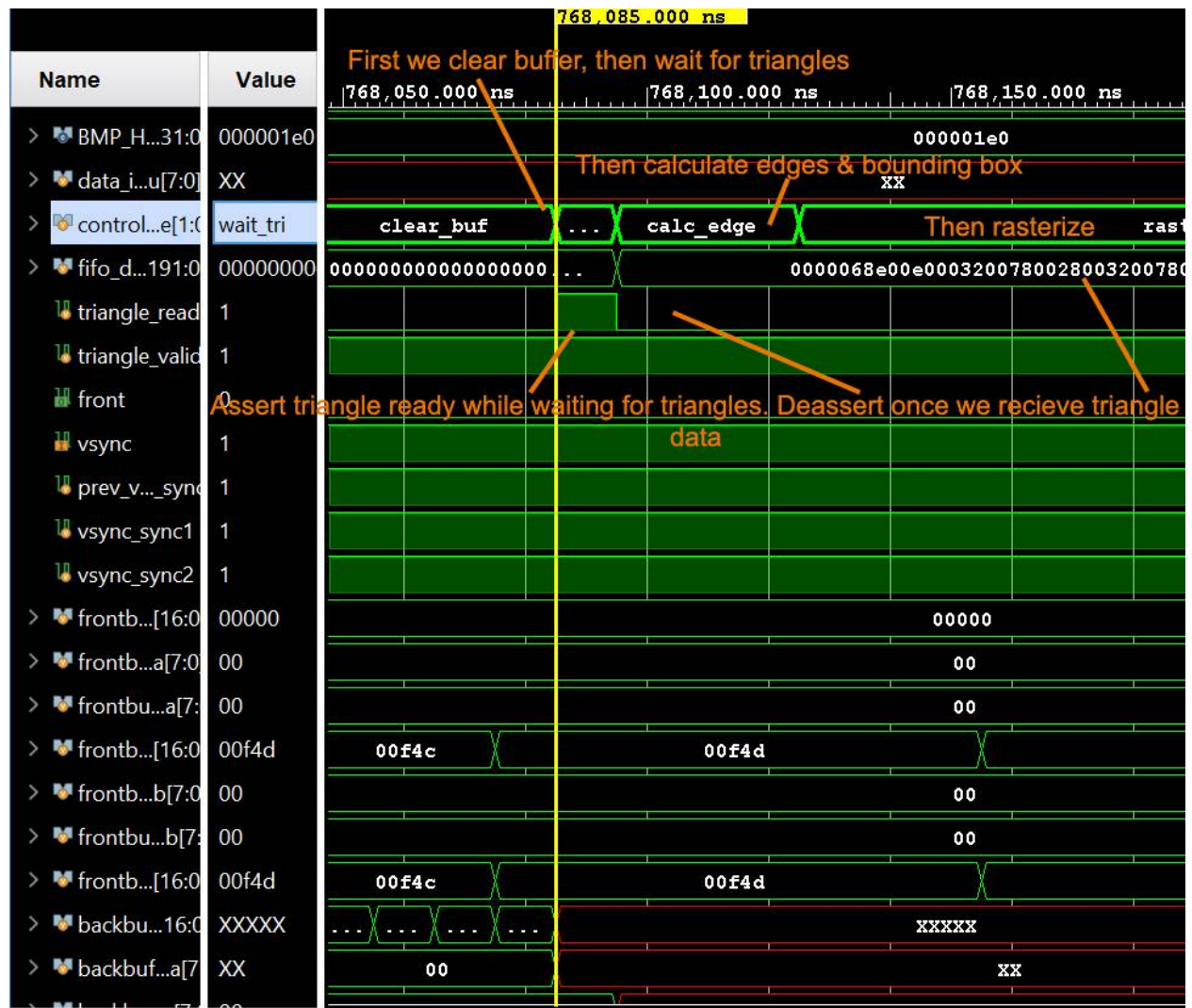
These different images display the world from the perspective of the aforementioned coordinates and camera orientations. We see that we successfully generate all triangles that might need to be drawn from a given point of view. In the last image, the small cube renders despite being fully behind the large cube. This is a result of z-buffering not being done in software but in hardware, and it is intended behavior.

Triangle Drawing

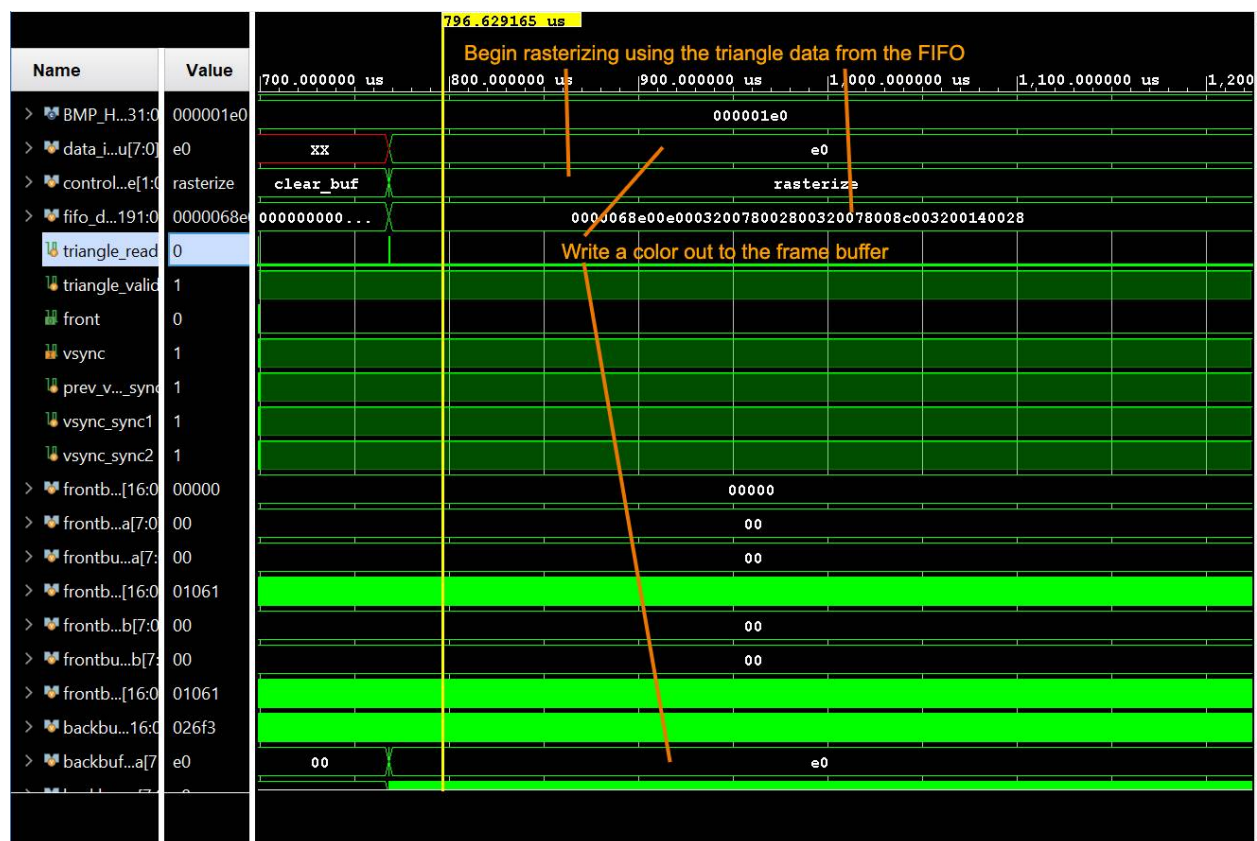
Buffer clearing annotated simulation:



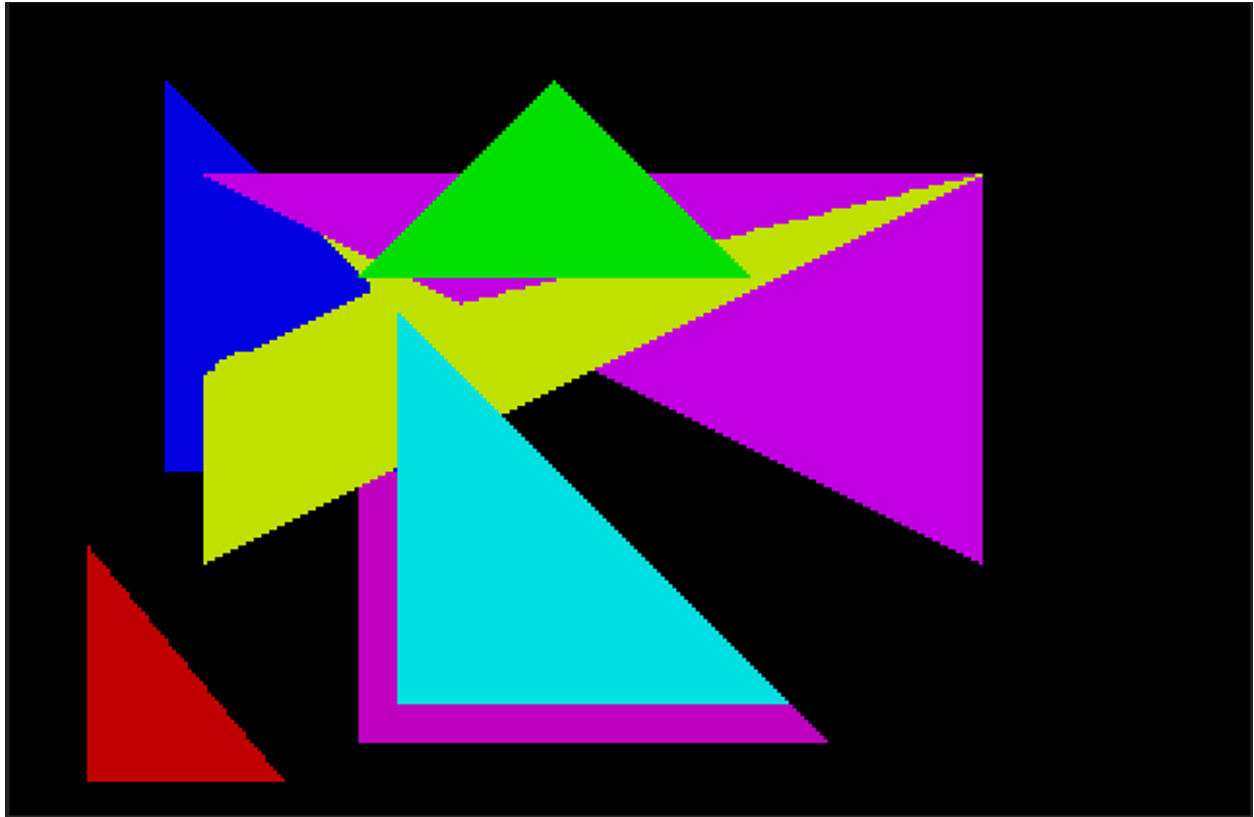
Handshaking annotated simulation:



Rasterizer writing frame buffer annotated simulation:



Result of axi_tb.sv:



Provided vertices:

```
// Triangle 1: Red (Bottom Left) - Farther Back (Z=50)
draw_triangle(
    9'd40, 8'd20,    // v1
    9'd140, 8'd120, // v2
    9'd40, 8'd120,  // v3
    8'hE0,          // Red (RGB332: 111_000_00)
    16'd50, 16'd50, 16'd50
);

// Triangle 2: Green (Top Middle) - Closer (Z=10). **FIXED CW/CCW Winding**
draw_triangle(
    9'd140, 8'd20,    // v1
    9'd190, 8'd70,    // v2 (Swapped from original to fix winding)
    9'd90, 8'd70,     // v3 (Swapped from original to fix winding)
    8'h1C,           // Green (RGB332: 000_111_00)
    16'd10, 16'd10, 16'd10
);
```



```

// Triangle 3: Blue (Top Left) - Z Gradient (Z=10 to 100)
draw_triangle(
    9'd20, 8'd140,
    9'd70, 8'd200,
    9'd20, 8'd200,
    8'h03,          // Blue (RGB332: 000_000_11)
    16'd10, 16'd100, 16'd10
);

// --- NEW OVERLAPPING TRIANGLES ---

// Triangle 4: Cyan (Overlapping Center) - **VERY CLOSE (Z=5)**
// This will be drawn LAST in the FIFO and should overwrite all others.
draw_triangle(
    9'd100, 8'd80,   // V1
    9'd200, 8'd180,  // V2
    9'd100, 8'd180,  // V3
    8'hFC,           // Yellow (RGB332: 111_111_00)
    16'd5, 16'd5, 16'd5
);

// Triangle 5: Magenta (Overlapping Yellow) - **VERY FAR (Z=100)**
// This will be drawn AFTER Yellow, but should be hidden by the Z-buffer.
draw_triangle(
    9'd90, 8'd70,    // V1
    9'd210, 8'd190,  // V2
    9'd90, 8'd190,   // V3
    8'hC3,           // Magenta (RGB332: 110_000_11)
    16'd100, 16'd100, 16'd100
);

draw_triangle(
    9'd50, 8'd300,   // V1 (Left)  Z=10
    9'd250, 8'd300,  // V2 (Right) Z=90
    9'd50, 8'd400,   // V3 (Left)  Z=10
    8'h1F,           // Cyan
    16'd10, 16'd90, 16'd10
);

// Triangle B: Magenta (Slopes Z: High -> Low)
draw_triangle(
    9'd50, 8'd300,   // V1 (Left)  Z=90

```

```

9'd250, 8'd300,    // V2 (Right) Z=10
9'd250, 8'd400,    // V3 (Right) Z=10
8'hE3,             // Magenta
16'd90, 16'd10, 16'd10
);

```

The yellow triangle has a z gradient (bottom vertex has very small Z, top edge has very large Z). It intersects with the other triangles on a per-pixel basis when the other triangles have a smaller z, showing working z-buffer functionality.

Design Resources and Statistics

LUTs	5,103
DSP	20
Memory (BRAM)	74
Flip-Flop	3989
Latches	0
Frequency	105.218 MHz
Static Power	0.076W
Dynamic Power	0.419W
Total Power	0.495W

Difficulties

Triangle Transformation

Overall, the software was by far easier to debug. Here are the biggest issues we faced when implementing software on the MicroBlaze.

1. We could not get the keyboard to work. We set up the SPI and USB hardware in the block design exactly as Lab 6 specified and used our working SPI functions from Lab 6. After debugging, we realized that `MAX3421E_init()` was failing somehow, as no code after it would ever run. From this point, we were not sure exactly how to debug other than by checking that our block design was correct. Thus, to showcase the dynamic rendering within the timeframe, we instead switched to creating a path for the camera to follow.
2. We were incorrectly using memory-mapped I/O within the MicroBlaze. For a while, we were using a struct like this:

```
typedef struct __attribute__((packed)) {  
    uint16_t vertices[9];  
    uint8_t color;  
    int32_t r_area;  
} DATA;
```

We figured that since in Lab 7, we were able to access the VRAM as an array of bytes, we should be able to do similarly here. However, it did not work, even with the packed attribute. Eventually, we switched to having a struct with only 32-bit fields. This ended up wasting quite a bit of time because we didn't know whether there was a problem with our AXI handshaking logic (as in Lab 7, this worked in simulation but not hardware), and assumed that it was the handshaking logic at fault.

3. We had many miscellaneous Vitis errors. The biggest problem was Vitis on Tanmay's computer throwing an error about an invalid platform, which meant he could not test on

hardware for a while. Zaheen eventually had similar issues and had to spend at least 2 hours debugging, and ended up having to change Makefiles in Vitis manually to get it to work.

4. One unexpected issue was that MicroBlaze cannot compute trigonometric functions even with the FPU enabled. It would silently fail (and stop all output, including previous outputs, to the serial terminal somehow) when called. This required us to consider approximating them instead, but we ultimately ended up using a size 256 lookup table to store sin values. This allowed us to skip computation while not using too much memory.
5. While developing the logic in the first place (before even running on MicroBlaze), we had an issue with the logic hanging forever. It turns out that this happened when w was close but not equal to 0, causing floating point errors with future divisions and creating NaNs.

Triangle Drawing

AXI

We had to perform a careful balancing act of deciding which information to compute on the microblaze and which information we could compute in hardware. One of the main things that was considered was the size of the packet we would send through AXI. The size we landed on was 192 bits, but we floated larger and smaller packets, thinking of how many clock cycles it would take each packet to transfer and how much the microblaze could handle. After trial and error we found the optimal amount of information for the microblaze to transmit to hardware: projected x,y,z of 3 vertices, inverse area, and color.

Clock domain crossing

At times, usage of multiple clocks in logic caused clock domain crossing issues, where when we tried detecting the falling edge of vsync in order to switch frame buffers, our logic often missed the edge, which ran on a clock 4 times slower. This resulted in us having to research and implement a clock domain crossing solution, which synchronized the pixel clock with the main

100 MHz clock in multiple areas of the code to ensure that edges were detected correctly and that we were able to swap our frame buffers and clear them appropriately.

Floating point issues

Many hours were spent debugging floating-point and bit-width issues, where each multiplication and addition operation had to be put into logic signals that were appropriately sized. In addition, we could not resize signals randomly, as sometimes the bulk of the significant data was in an arbitrary location.

For example, to fix a 71-bit multiplication problem, we tried to resize one of our products to 46 bits, and therefore resize one weight to 30 bits. Although this design could now fit inside a DSP48 module, it turned out that many significant bits for the barycentric weights for triangles in the ranges of z between 100-250 were excluded with this operation. This resulted in very inaccurate triangle drawing for specific areas of the 3d mesh, while other areas were drawn fine. These issues were extremely difficult to track and test for, causing many hours of debugging, trial and error.

Fixing these issues required extending precision, and therefore product widths in DSPs and trying to earn time back in other areas to ensure we still drew enough triangles to render a scene.

Resource usage

During the entire project, we were approaching the hardware limit of BRAM usage. With 2 8-bit frame buffers sized 76.8 kB, a z -buffer sized 76.8 kB, a 192-bit wide FIFO, a microblaze, and overhead, we received resource overutilization errors multiple times, having to redesign parts of the project to cut usage.

Pipelining and timing

Our multiplications required DSP usage in order to avoid failing timing and having an unsustainable worst negative slack. In order to achieve this, we had to build a non-linear pipeline where we stored each multiplication operand and product in a pipeline register to

encourage the synthesizer to use a DSP. Even still, we had multiple errors to deal with, including DRC violations due to the very wide (71-bit) multiplications that some of our calculations were forced to perform. Fixing these errors and allowing the DSP modules to synthesize correctly was difficult. In the end, however, we were able to achieve a fantastic worst negative slack of 0.6 nS, which was in fact within the 10 nS clock period, ensuring no issues with setup time violations.

Conclusion

Despite having faced many difficulties, we were able to get partial functionality. Everything seemed to work except our z-buffering, which failed on hardware but worked on simulation. It is possible we were not thorough enough in the simulation. Given that it seemed full functionality was right around the corner, we were very disappointed with our final demo. After at least 110 hours of work combined between both of us, we thought we would have more to show. We did work a little more that night and reduced the glitching by increasing precision, but we had to turn in our boards. Thus, we can no longer test on hardware either. However, we still wish to work on our own time to finish and add more features on our own time.